操作系统实验报告

Lab 1: The Trouble with Concurrent Programming

姓　　名：李聪

学　　号：22920212204124

院　　系：信息学院人工智能系

专　　业：人工智能

年　　级：2021 级

指导教师：曹冬林

2023 年 11 月 2 日

一、实验目的

1. 配置并熟悉 Nachos；
2. 初步体验 Nachos 下并发式程序设计。

二、实验要求

2.1 实现优先级排序双向链表

编写一个 C++程序，以根据以下定义实现双向链表：

```
class DLLElement
{
public:
    DLLElement(void *itemPtr, int sortKey); // initialize a list element
    DLLElement *next;                       // next element on list
                                            // NULL if this is the last
    DLLElement *prev;                       // previous element on list
                                            // NULL if this is the first
    int key;                                // priority, for a sorted list
    void *item;                             // pointer to item on the list
};
class DLList
{
public:
    DLList();                 // initialize the list
    ~DLList();                // de-allocate the list
    void Prepend(void *item);  // add to head of list (set key = min_key-1)
    void Append(void *item);   // add to tail of list (set key = max_key+1)
    void *Remove(int *keyPtr); // remove from head of list
                               // set *keyPtr to key of the removed item
                               // return item (or NULL if list is empty)
    bool IsEmpty();            // return true if list has elements
    // routines to put/get items on/off list in order (sorted by key)
    void SortedInsert(void *item, int sortKey);
    void *SortedRemove(int sortKey); // remove first item with key==sortKey
    // return NULL if no such item exists
private:
    DLLElement *first; // head of the list, NULL if empty
    DLLElement *last;  // last element of the list, NULL if empty
};
```

双向链表应根据 key 对链表元素进行排序（对于没有键参数的插入操作，请为该操作指定一致的键值）。编写程序，使其代码包含在三个文件中：dllist.h、dllist.cc、dllis-driver.cc。

前两个文件应该提供上述两个类的定义和实现，第三个文件包含两个函数，其中一个使用随机键生成 N 个链表元素（或者为了帮助调试，你可以用更仔细选择的键顺序控制输入序列）并将它们插入到双向链表中，另一个删除 N

个链表元素从列表的头部开始，并将已删除的项目打印到控制台。这两个函数都应将整数 N 和指向列表的指针作为参数。

若要验证是否确实正确实现了类和驱动程序函数，请创建一个包含程序 main 函数的单独文件。在此函数中，首先分配列表，然后调用上面的驱动程序函数，为参数传入适当的值。您需要演示的行为是 remove 函数完全删除您按排序顺序插入的项目。您还应该执行其他测试来验证您的实现是否实现了双向链表。

2.2 熟悉 Nachos 并了解它的线程系统

具体而言，这部分作业要求您执行以下步骤：

（1）将 dllist.h、dllist.cc 和 dllist-driver.cc 文件复制到 threads 子目录中。修改根 nachos 目录中 Makefile.common 中 THREAD_H 和 THREAD_C 的定义，以包含这些文件并更新 Makefile 依赖项。

（2）创建一个类似于文件 threadtest.cc 的驱动程序文件，该文件使用 DLList-driver.cc 中的函数调用 DLList 类对 threads/main.cc 进行更改，以便执行 nachos 命令时会执行新驱动程序文件中的函数，而不是文件 threadtest.cc 中的函数 threadtest。

（3）修改 DLList 类和 ThreadTest，以强制交错显示有趣的错误或意外行为。您应该能够在演示过程中使用命令行标志枚举和演示每个场景，而无需重新编译测试程序。

（4）考虑一下您可以演示的 bug 行为，并将它们划分为几个类别。您的写作应该描述每一类 bug，概述可能导致 bug 发生的交错，并解释交错是如何导致结果行为的。注意，如果不同的线程表现出不同的交织行为，您可能会看到更有趣的行为。您对 bug 的处理应该是彻底的，但不一定是详尽无遗的。尽量把注意力集中在最“有趣”的交错处，避免花时间描述或演示本质上相似的行为。这里的目标是显示您理解并发行为，而不是为您或我们创建大量繁忙的工作。

三、实验流程

3.1 实现优先级排序双向链表

3.1.1 编写相关的 dllist.h、dllist.cc、dllis-driver.cc 文件

为了方便验证是否成功实现两个类和两个驱动函数，在 DLList 类中增加 show 函数，以便查看链表的所有元素。

3.1.2 修改 main.cc 文件

引入头文件“dllist.h”

```
#include "dllist.h"
```

声明 dllis-driver.cc 中的函数

```
extern void insert(int n, DLList &dllist);
extern void remove(int n, DLList &dllist);
```

将 main 函数中 THREADS 部分注释掉

```
#ifdef THREADS
    // for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount) {
    //   argCount = 1;
    //   switch (argv[0][1]) {
    //   case 'q':
    //     testnum = atoi(argv[1]);
```

```
    //     argCount++;
    //     break;
    //   default:
    //     testnum = 1;
    //     break;
    //   }
    // }

    // ThreadTest();
#endif
```

在 main 函数中，添加测试代码

```
    DLList dllist;
    dllist.show();
    // 当链表为空时，调用 Prepend
    dllist.Prepend(NULL);
    dllist.show();
    // 当链表非空时，调用 Prepend
    dllist.Prepend(NULL);
    dllist.show();
    // 当链表非空时，调用 Remove
    int key = -1;
    dllist.Remove(&key);
    printf("List remove %d.\n", key);
    dllist.show();
    dllist.Remove(&key);
    printf("List remove %d.\n", key);
    dllist.show();
    // 当链表为空时，调用 Remove
    dllist.Remove(&key);
    dllist.show();
    // 当链表为空时，调用 Append
    dllist.Append(NULL);
    dllist.show();
    // 当链表非空时，调用 Append
    dllist.Append(NULL);
    dllist.show();
    // 清空链表
    while(!dllist.IsEmpty()) dllist.Remove(&key);
    dllist.show();
    // 当链表为空时，调用 SortedInsert 插入
    dllist.SortedInsert(NULL, 50);
    dllist.show();
    // 调用 SortedInsert，头插入
    dllist.SortedInsert(NULL, 0);
```

```
    dllist.show();
    // 调用 SortedInsert，尾插入
    dllist.SortedInsert(NULL, 100);
    dllist.show();
    // 调用 SortedInsert，中间插入
    dllist.SortedInsert(NULL, 50);
    dllist.show();
    // 调用 SortedRemove，删除不存在的 key
    dllist.SortedRemove(20);
    dllist.show();
    // 调用 SortedRemove，中间删除
    dllist.SortedRemove(50);
    dllist.show();
    // 调用 SortedRemove，头删除
    dllist.SortedRemove(0);
    dllist.show();
    // 调用 SortedRemove，尾删除
    dllist.SortedRemove(100);
    dllist.show();
    // 清空链表
    while(!dllist.IsEmpty()) dllist.Remove(&key);
    dllist.show();
    // 调用 insert
    insert(5, dllist);
    dllist.show();
    // 调用 remove
    remove(5, dllist);
    dllist.show();
```

3.1.3 修改/codo/Makefile.common 文件中的 THREAD_H、THREAD_C、THREAD_O

```
THREAD_H =../threads/copyright.h\
    ../threads/list.h\
    ../threads/scheduler.h\
    ../threads/synch.h \
    ../threads/synchlist.h\
    ../threads/system.h\
    ../threads/thread.h\
    ../threads/utility.h\
    ../threads/dllist.h\
    ../machine/interrupt.h\
    ../machine/sysdep.h\
    ../machine/stats.h\
    ../machine/timer.h
THREAD_C =../threads/main.cc\
    ../threads/list.cc\
```

```
    ../threads/scheduler.cc\
    ../threads/synch.cc \
    ../threads/synchlist.cc\
    ../threads/system.cc\
    ../threads/thread.cc\
    ../threads/utility.cc\
    ../threads/threadtest.cc\
    ../threads/dllist.cc\
    ../threads/dllist-driver.cc\
    ../machine/interrupt.cc\
    ../machine/sysdep.cc\
    ../machine/stats.cc\
    ../machine/timer.cc
THREAD_O =main.o list.o scheduler.o synch.o synchlist.o system.o thread.o \
    utility.o threadtest.o dllist.o dllist-driver.o interrupt.o stats.o
sysdep.o timer.o
```

3.2 熟悉 Nachos 并了解它的线程系统

3.2.1 编写 yield.cc 文件

3.2.2 修改 threadtest.cc 文件

3.2.3 修改 dllist-driver.cc、dllist.cc 和 threadtest.cc，添加 YIELD 的调用

3.2.4 添加调试信息

函数 void DEBUG(char flag, char *format, ...)是用来输出调式信息的，flag 表示调试的类型，后面的(char *format, ...)和 printf 的用法相同。用户可以自己设定 flag，我设定的 flag 是‘e’，需要调试时输入参数“-d e”。

3.2.5 修改 main.cc

添加外部函数的声明

```
extern void YieldInit(int place);
extern void YIELD(int place);
extern void ThreadInit(int t, int n);
```

这些函数的作用可以在 3.2.1 和 3.2.2 节中找到。

修改 THREADS 部分

```
#ifdef THREADS
    for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount)
    {
        argCount = 1;
        if (!strcmp(*argv, "-q"))
        {
            if(argc >= 3 && argv[1][0] != '-')
            ThreadInit(atoi(argv[1]), atoi(argv[2]));
            argCount += 2;
        }
        else if(!strcmp(*argv, "-y"))
```

```
        {
            YieldInit(atoi(argv[1]));
            ++ argCount;
        }
    }
    ThreadTest();
#endif
```

该部分实现了对指令参数的识别与对应操作的执行，最后调用“ThreadTest()”。

3.3.6 命令：./nachos [-q [T N]] [-d [flags]] [-y place]

四、测试及分析

4.1 验证双向链表的正确性

```
List is empth!
List from head to tail: 50
List from tail to haed: 50
List from head to tail: 49 50
List from tail to haed: 50 49
List remove 49.
List from head to tail: 50
List from tail to haed: 50
List remove 50.
List is empth!
List is empth!
List from head to tail: 50
List from tail to haed: 50
List from head to tail: 50 51
List from tail to haed: 51 50
List is empth!
List from head to tail: 50
List from tail to haed: 50
List from head to tail: 0 50
List from tail to haed: 50 0
List from head to tail: 0 50 100
List from tail to haed: 100 50 0
List from head to tail: 0 50 50 100
List from tail to haed: 100 50 50 0
List from head to tail: 0 50 50 100
List from tail to haed: 100 50 50 0
List from head to tail: 0 50 100
List from tail to haed: 100 50 0
List from head to tail: 50 100
List from tail to haed: 100 50
List from head to tail: 50
```

```
List from tail to haed: 50
List is empth!
List from head to tail: 21 25 30 41 64
List from tail to haed: 64 41 30 25 21
List is empth!
```

4.2 并发错误分析

4.2.1 重复释放内存

```
Yield in 12.
```

当线程 1 插入的元素都比线程 0 插入的最小的两个数大时，

线程 0：正常插入，直至要删除第一个元素，将 temp 指向第二个元素，转到线程 1；

线程 1：正常插入，直至要删除第一个元素，将 temp 指向第二个元素，转到线程 0；

线程 0：释放第一个结点的空间，将 first 指向 temp，继续准备删除第二个结点，将 temp 指向第三个结点，转到线程 1；

线程 1：释放第二个结点的空间，将 first 指向 temp（下图中的 temp 1），即 first 仍指向第二个结点，temp 1 指向第三个结点，转到线程 0；

线程 0：释放 first 指向的结点，即第二个结点的空间，重复释放发生错误。

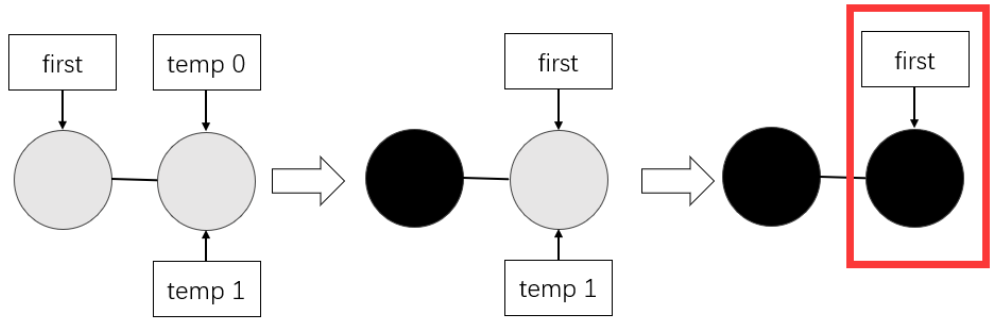

图 4.2.1-1

输出信息：

```
[ai22920212204124@mcore threads]$ ./nachos -d e -y 12
thread 0 is running.
thread 0: insert key:33, item:0x8caa160.
thread 0: insert key:24, item:0x8cb0218.
thread 0: insert key:21, item:0x8cb0240.
thread 0: insert key:82, item:0x8cb0268.
thread 0 yield in 12.
thread 1 is running.
thread 1: insert key:38, item:0x8cb0290.
thread 1: insert key:43, item:0x8cb02b8.
thread 1: insert key:28, item:0x8cb02e0.
thread 1: insert key:68, item:0x8cb0308.
thread 1 yield in 12.
thread 0: remove key:21, item:0x8cb0240.
```

```
thread 0 yield in 12.
thread 1: remove key:21, item:0x8cb0240.
thread 1 yield in 12.
*** glibc detected *** ./nachos: double free or corruption
(fasttop): 0x08cb0228 ***
======= Backtrace: =========
/lib/libc.so.6[0x76aa15]
/lib/libc.so.6(cfree+0x59)[0x76ea89]
/usr/lib/libstdc++.so.6(_ZdlPv+0x21)[0xa4c5c1]
./nachos[0x804b034]
./nachos[0x804b13d]
./nachos[0x804a751]
./nachos[0x804c5dc]
./nachos[0x804c5d4]
======= Memory map: ========
006e2000-006fd000 r-xp 00000000 fd:00 22970468
/lib/ld-2.5.so
006fd000-006fe000 r-xp 0001a000 fd:00 22970468
/lib/ld-2.5.so
006fe000-006ff000 rwxp 0001b000 fd:00 22970468
/lib/ld-2.5.so
00701000-00857000 r-xp 00000000 fd:00 22970383
/lib/libc-2.5.so
00857000-00859000 r-xp 00156000 fd:00 22970383
/lib/libc-2.5.so
00859000-0085a000 rwxp 00158000 fd:00 22970383
/lib/libc-2.5.so
0085a000-0085d000 rwxp 0085a000 00:00 0
00882000-008a9000 r-xp 00000000 fd:00 22970390
/lib/libm-2.5.so
008a9000-008aa000 r-xp 00026000 fd:00 22970390
/lib/libm-2.5.so
008aa000-008ab000 rwxp 00027000 fd:00 22970390
/lib/libm-2.5.so
00999000-00a79000 r-xp 00000000 fd:00 21339615
/usr/lib/libstdc++.so.6.0.8
00a79000-00a7d000 r-xp 000df000 fd:00 21339615
/usr/lib/libstdc++.so.6.0.8
00a7d000-00a7e000 rwxp 000e3000 fd:00 21339615
/usr/lib/libstdc++.so.6.0.8
00a7e000-00a84000 rwxp 00a7e000 00:00 0
00a86000-00a91000 r-xp 00000000 fd:00 22971146
/lib/libgcc_s-4.1.2-20080825.so.1
```

```
00a91000-00a92000 rwxp 0000a000 fd:00 22971146
/lib/libgcc_s-4.1.2-20080825.so.1
08048000-0804f000 r-xp 00000000 fd:00 16879234
/home/ai2021/ai22920212204124/nachos-3.4/code/threads/nachos
0804f000-08050000 rwxp 00006000 fd:00 16879234
/home/ai2021/ai22920212204124/nachos-3.4/code/threads/nachos
08ca4000-08cc5000 rwxp 08ca4000 00:00 0
[heap]
f7f45000-f7f47000 rwxp f7f45000 00:00 0
f7f57000-f7f58000 rwxp f7f57000 00:00 0
ff9c9000-ff9de000 rwxp 7ffffffe9000 00:00 0
[stack]
ffffe000-fffff000 r-xp ffffe000 00:00 0
```

已放弃

4.2.2 尾指针为 NULL，但头指针不为 NULL，在链表末尾插入，造成非法访问

Yield in 26

线程 0 在第一次插入时，因为双链表为空，为 first 赋值后，转到线程 1；

线程 1 在链表尾插入时，因为头尾指针不同时为 0，认为链表非空，但由于尾指针为 NULL，造成非法访问。

```
else if(ptr == NULL) // insert in the tail
{
    YIELD(37);
    elm->prev = last;
    YIELD(38);
    last->next = elm;
    YIELD(39);
    last = elm;
    YIELD(40);
}
```

图 4.2.2-1

输出信息：

```
[ai22920212204124@mcore threads]$ ./nachos -d e -y 26
thread 0 is running.
thread 0 yield in 26.
thread 1 is running.
[ERROR]first is not NULL, last is NULL!
thread 1: insert key: 6, item:0x85aa218.
[ERROR]first is not NULL, last is NULL!
thread 1: insert key:30, item:0x85aa240.
[ERROR]first is not NULL, last is NULL!
```

段错误

4.2.3 结点位置插入错误

Yield in 29.

当线程要插入一个比头节点小的结点，且下一个线程也要插入比头节点小，但是比该线程的节点大时。

当前头节点 key=52 时，线程 0 准备插入 key=45，将 ptr 0 指向 52，转到线程 1；
线程 1，准备插入 key=23，将 ptr 1 指向 52，转到线程 0；
线程 0，从 ptr 0 开始寻找可插入位置，即头插入，更新 first，准备下一次插入，转到线程 1；
线程 1，从 ptr 1 开始寻找可插入位置，在 52 前，因为 ptr 1 不等于 first，且不为 NULL，所以执行中间插入，就造成了 45，23，52 的错误。

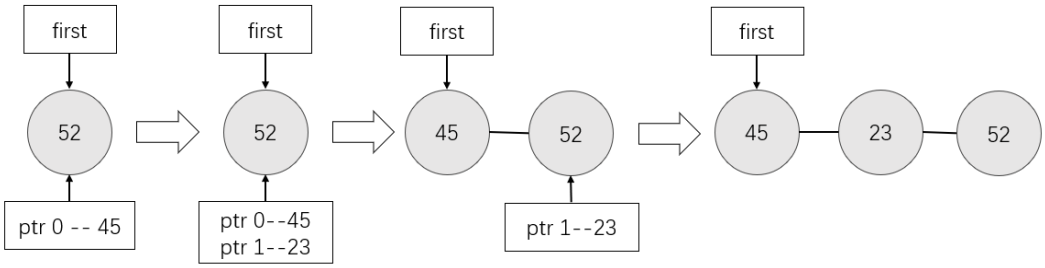

图 4.2.3-1

输出信息：

```
[ai22920212204124@mcore threads]$ ./nachos -d e -y 29
thread 0 is running.
thread 0: insert key:52, item:0x9741160.
thread 0 yield in 29.
thread 1 is running.
thread 1 yield in 29.
thread 0: insert key:45, item:0x9747218.
thread 0 yield in 29.
thread 1: insert key:23, item:0x9747240.
thread 1 yield in 29.
thread 0: insert key: 0, item:0x973b080.
thread 0 yield in 29.
thread 1: insert key:33, item:0x973b0a8.
thread 1 yield in 29.
thread 0: insert key:94, item:0x973b0d0.
thread 0: remove key: 0, item:0x973b080.
thread 0: remove key:33, item:0x973b0a8.
thread 0: remove key:45, item:0x9747218.
thread 0: remove key:23, item:0x9747240.
thread 0 is finish.
List from head to tail:
key: 52, item: 0x9741160
key: 94, item: 0x973b0d0
List from tail to haed:
key: 94, item: 0x973b0d0
key: 52, item: 0x9741160
thread 1: insert key:10, item:0x9747280.
```

```
thread 1 yield in 29.
thread 1: insert key: 0, item:0x97471f0.
thread 1: remove key: 0, item:0x97471f0.
thread 1: remove key:52, item:0x9741160.
thread 1: remove key:94, item:0x973b0d0.
thread 1: remove key:10, item:0x9747280.
thread 1 is finish.
List is empty!
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 120, idle 0, system 120, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0
```

4.2.4 释放头节点后，未及时更新头指针时，另一个线程插入时，发生错误

Yield in 13，14，15，16.

执行完 delete first 后，在这些位置转到其他线程，其他线程执行插入操作，都要从 first 开始，而 first 还未更新，造成程序卡死。

```
delete first;
YIELD(13);
if(temp == NULL)
{
    YIELD(14);
    last = NULL;
}
else
{
    YIELD(15);
    temp->prev = NULL;
}
YIELD(16);
first = temp;
```

图 4.2.4-1

输出信息：

```
[ai22920212204124@mcore threads]$ ./nachos -d e -y 13
thread 0 is running.
thread 0: insert key:73, item:0x971f160.
thread 0: insert key:13, item:0x9725218.
thread 0: insert key:21, item:0x9725240.
thread 0: insert key:44, item:0x9725268.
thread 0 yield in 13.
thread 1 is running.
```

```
thread 1: insert key:60, item:0x9725290.

[3]+  Stopped                 ./nachos -d e -y 13
```

4.2.5 双链表出现“两个”头节点

```
Yield in 35.
```

只要两个线程中都有头插法存在，就会发生错误。

线程 0，在头节点为 53 时，插入 24，在更新 first 前，转到线程 1；

线程 1，插入 16，在更新 first 前，转到线程 0；

线程 0，更新 first 指向 24，此时双链表已经发生了错误。

在例子中，实际上 16 和 53 结点之间还有一个 35 结点，但是这影响不大，在线程 0 完成插入并删除后，转到线程 1，此时双链表是没有 last 指针，虽然还可以安全插入 18，但是只要在尾巴插入就会出现段错误，即便没有段错误，最后删除链表的最后一个元素，也会造成非法访问。

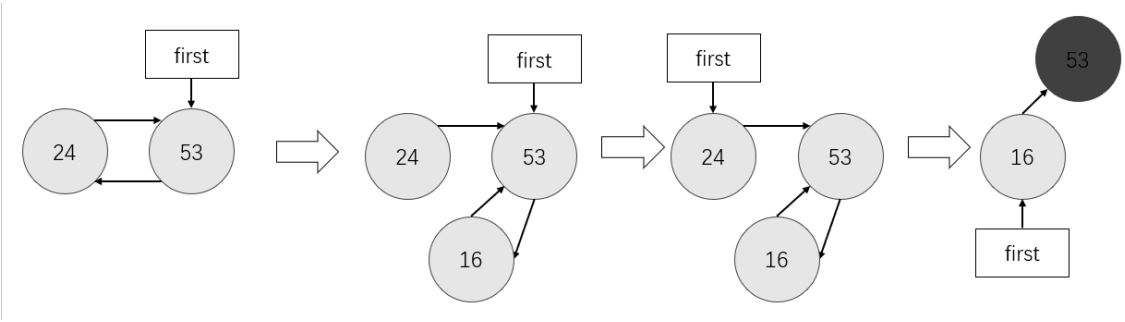

图 4.2.5-1

输出信息：

```
[ai22920212204124@mcore threads]$ ./nachos -d e -y 35
thread 0 is running.
thread 0: insert key:53, item:0x9c44160.
thread 0: insert key:77, item:0x9c4a218.
thread 0 yield in 35.
thread 1 is running.
thread 1: insert key:62, item:0x9c4a268.
thread 1 yield in 35.
thread 0: insert key:24, item:0x9c4a240.
thread 0: insert key:35, item:0x9c3e080.
thread 0: remove key:24, item:0x9c4a240.
thread 0: remove key:53, item:0x9c44160.
thread 0: remove key:62, item:0x9c4a268.
thread 0: remove key:77, item:0x9c4a218.
thread 0 is finish.
List is empty!
thread 1: insert key:16, item:0x9c4a290.
[ERROR]first is not NULL, last is NULL!
thread 1: insert key:18, item:0x9c4a1f0.
[ERROR]first is not NULL, last is NULL!
```

段错误

Yield in 39. 会由于和上面类似的原因出现“两个”尾结点的错误。

4.2.6 插入位置被删除

Yield in 33.

当一个线程要执行头插法，并转到另一个线程，另一个线程直接执行到结束，清空链表，first 等于 NULL，调用 first->prev 出现段错误。

输出信息：

```
[ai22920212204124@mcore threads]$ ./nachos -d e -y 33
thread 0 is running.
thread 0: insert key:30, item:0x8493160.
thread 0 yield in 33.
thread 1 is running.
thread 1 yield in 33.
thread 0: insert key:18, item:0x8499218.
thread 0: insert key:62, item:0x848d080.
thread 0: insert key:33, item:0x848d0a8.
thread 0: remove key:18, item:0x8499218.
thread 0: remove key:30, item:0x8493160.
thread 0: remove key:33, item:0x848d0a8.
thread 0: remove key:62, item:0x848d080.
thread 0 is finish.
List is empty!
```

Yield in 34. 同上。

Yield in 37,38.是尾插入时，类似的错误。

Yield in 41,42,43,44. 是在链表中间插入时，类似的错误。

五、实验总结

5.1 srand 使用不当

实验使用 rand 产生随机数，刚开始使用 srand(time(0))分别在每个线程（SimpleThread）里设置不同的种子，但是由于线程运行的过快小于 1s，使得不同线程插入的数都是一样的随机性大大下降。最终将 srand(time(0))移到 ThreadTest 中，使所用线程共用一个种子，问题得到解决。

5.2 线程名字创建困难

C 语言不能像 python 语言那样方便的将整数转化为字符串，通过查阅资料发现了 int sprintf(char *buffer, const char *format [, argument,…])函数可以将格式化输出到字符串中，实现了创建线程名字的需求。

六、实验心得

本次实验详细地分析了线程切换是如何影响程序的运行的，通过总结错误的种类，使我熟悉了线程的相关知识。了解了线程冲突，也尝试编写更好的代码，减少冲突，提升了代码的质量。

在实验中，通过阅读实验代码，学习到了一些有用的编程技巧，比如说将一些函数需要的特定变量局限在文件内，然后通过别的文件的调用初始化函数，进行初始化，这样做能够减少变量重名以及对其他文件的影响；还有利用条件编译语句防止重复编译的方法；以及对命令行参数的处理及使用的方法。

初略地了解了在终端中编译运行程序的方法，对相关配置文件有了一定的了解。

七、附件

附件 1 dllist.h

```
#ifndef DLLIST_H
#define DLLIST_H
class DLLElement
{
public:
    DLLElement(void *itemPtr, int sortKey); // initialize a list element
    DLLElement *next;                       // next element on list
                                            // NULL if this is the last
    DLLElement *prev;                       // previous element on list
                                            // NULL if this is the first
    int key;                                // priority, for a sorted list
    void *item;                             // pointer to item on the list
};
class DLList
{
public:
    DLList();                 // initialize the list
    ~DLList();                // de-allocate the list
    void Prepend(void *item);  // add to head of list (set key = min_key-1)
    void Append(void *item);   // add to tail of list (set key = max_key+1)
    void *Remove(int *keyPtr); // remove from head of list
                               // set *keyPtr to key of the removed item
                               // return item (or NULL if list is empty)
    bool IsEmpty();            // return true if list has elements
    // routines to put/get items on/off list in order (sorted by key)
    void SortedInsert(void *item, int sortKey);
    void *SortedRemove(int sortKey); // remove first item with key==sortKey
    // return NULL if no such item exists
    void show(); // show the element of list
private:
    DLLElement *first; // head of the list, NULL if empty
    DLLElement *last;  // last element of the list, NULL if empty
};
#endif
```

附件 2 dllist.cc

```
#include <iostream>
#include "dllist.h"
#define KEY 50
```

```
4
5   DLLElement::DLLElement(void * itemPtr, int sortKey)
6   {
7       next = NULL;
8       prev = NULL;
9       key = sortKey;
10      item = itemPtr;
11  }
12
13  DLList::DLList()
14  {
15      first = NULL;
16      last = NULL;
17  }
18
19  DLList::~DLList()
20  {
21      int key;
22      while(!IsEmpty())
23      {
24          Remove(&key);
25      }
26  }
27
28  void DLList::Prepend(void *item)
29  {
30      if(IsEmpty())
31      {
32          DLLElement *elm = new DLLElement(item, KEY);
33          first = elm;
34          last = elm;
35      }
36      else
37      {
38          DLLElement *elm = new DLLElement(item, first->key - 1);
39          elm->next = first;
40          first->prev = elm;
41          first = elm;
42      }
43  }
44
45  void DLList::Append(void *item)
```

```
{

    if(IsEmpty())
    {
        DLLElement *elm = new DLLElement(item, KEY);
        first = elm;
        last = elm;
    }
    else
    {
        DLLElement *elm = new DLLElement(item, last->key + 1);
        elm->prev = last;
        last->next = elm;
        last = elm;
    }
}

void *DLList::Remove(int *keyPtr)
{
    if(IsEmpty())
    {
        *keyPtr = -1;
        return NULL;
    }
    else
    {
        void *item = first->item;
        *keyPtr = first->key;
        DLLElement * temp = first->next;
        delete first;
        if(temp == NULL)
        {
            last = NULL;
        }
        else
        {
            temp->prev = NULL;
        }
        first = temp;
        return item;
    }
}
```

```
88
89  bool DLList::IsEmpty()
90  {
91      if(first == NULL && last == NULL)
92          return true;
93      return false;
94  }
95
96  void DLList::SortedInsert(void *item, int sortKey)
97  {
98      DLLElement *elm = new DLLElement(item, sortKey);
99      if(IsEmpty())
100     {
101         first = elm;
102         last = elm;
103     }
104     else
105     {
106         DLLElement *ptr = first;
107         while(ptr != NULL && ptr->key < sortKey)
108         {
109             ptr = ptr->next;
110         }
111         if(ptr == first) // insert in the head
112         {
113             elm->next = first;
114             first->prev = elm;
115             first = elm;
116         }
117         else if(ptr == NULL) // insert in the tail
118         {
119             elm->prev = last;
120             last->next = elm;
121             last = elm;
122         }
123         else
124         {
125             elm->next = ptr;
126             ptr->prev->next = elm;
127             elm->prev = ptr->prev;
128             ptr->prev = elm;
129         }
```

```
    }
}

void *DLList::SortedRemove(int sortKey)
{
    DLLElement *ptr = first;
    while(ptr != NULL && ptr->key != sortKey)
    {
        ptr = ptr->next;
    }
    if(ptr == NULL)
    {
        return NULL;
    }
    void *item = ptr->item;
    if(ptr == first)
    {
        first = first->next;
        if(first == NULL) // list is empty
        {
            last = NULL;
        }
        else
        {
            first->prev = NULL;
        }
    }
    else if(ptr == last)
    {
        last = last->prev;
        if(last == NULL) // list is empty
        {
            first = NULL;
        }
        else
        {
            last->next = NULL;
        }
    }
    else
    {
        ptr->next->prev = ptr->prev;
```

```
        ptr->prev->next = ptr->next;
    }
    delete ptr;
    return item;
}

void DLList::show()
{
    if(IsEmpty())
    {
        printf("List is empth!\n");
    }
    else
    {
        DLLElement *tmp = first;
        printf("List from head to tail:\n");
        while(tmp != NULL)
        {
            printf("key: %d, item: %p\n", tmp->key, tmp->item);
            tmp = tmp->next;
        }
        tmp = last;
        printf("List from tail to head:\n");
        while(tmp != NULL)
        {
            printf("key: %d, item: %p\n", tmp->key, tmp->item);
            tmp = tmp->prev;
        }
    }
}
```

附件 3 dllist-driver.cc

```
#include <stdlib.h>
#include <time.h>
#include "dllist.h"

#define MAX_KEY 100

void insert(int n, DLList &dllist)
{
    srand(time(0));
    for(int i = 0; i < n; ++ i)
    {
```

```
        void *item = new int[1];
        dllist.SortedInsert(item, rand() % MAX_KEY);
    }
}

void remove(int n, DLList &dllist)
{
    for(int i = 0; i < n; ++ i)
    {
        int key = -1;
        dllist.Remove(&key);
    }
}
```

附件 4 main.cc

```
// main.cc
//  Bootstrap code to initialize the operating system kernel.
//
//  Allows direct calls into internal operating system functions,
//  to simplify debugging and testing.  In practice, the
//  bootstrap code would just initialize data structures,
//  and start a user program to print the login prompt.
//
//  Most of this file is not needed until later assignments.
//
// Usage: nachos -d <debugflags> -rs <random seed #>
//      -s -x <nachos file> -c <consoleIn> <consoleOut>
//      -f -cp <unix file> <nachos file>
//      -p <nachos file> -r <nachos file> -l -D -t
//              -n <network reliability> -m <machine id>
//              -o <other machine id>
//              -z
//
//    -d causes certain debugging messages to be printed (cf. utility.h)
//    -rs causes Yield to occur at random (but repeatable) spots
//    -z prints the copyright message
//
//  USER_PROGRAM
//    -s causes user programs to be executed in single-step mode
//    -x runs a user program
//    -c tests the console
//
//  FILESYS
//    -f causes the physical disk to be formatted
```

```
//    -cp copies a file from UNIX to Nachos
//    -p prints a Nachos file to stdout
//    -r removes a Nachos file from the file system
//    -l lists the contents of the Nachos directory
//    -D prints the contents of the entire file system
//    -t tests the performance of the Nachos file system
//
//  NETWORK
//    -n sets the network reliability
//    -m sets this machine's host id (needed for the network)
//    -o runs a simple test of the Nachos network software
//
//  NOTE -- flags are ignored until the relevant assignment.
//  Some of the flags are interpreted here; some in system.cc.
//
// Copyright (c) 1992-1993 The Regents of the University of California.
// All rights reserved.  See copyright.h for copyright notice and
limitation
// of liability and disclaimer of warranty provisions.

#define MAIN
#include "copyright.h"
#undef MAIN

#include "utility.h"
#include "system.h"
#include "dllist.h"

#ifdef THREADS
extern int testnum;
#endif

// External functions used by this file

extern void ThreadTest(void), Copy(char *unixFile, char *nachosFile);
extern void Print(char *file), PerformanceTest(void);
extern void StartProcess(char *file), ConsoleTest(char *in, char *out);
extern void MailTest(int networkID);

extern void insert(int n, DLList &dllist);
extern void remove(int n, DLList &dllist);

```

```
72  //----------------------------------------------------------------------
73  // main
74  //  Bootstrap the operating system kernel.
75  //
76  //  Check command line arguments
77  //  Initialize data structures
78  //  (optionally) Call test procedure
79  //
80  //  "argc" is the number of command line arguments (including the name
81  //      of the command) -- ex: "nachos -d +" -> argc = 3
82  //  "argv" is an array of strings, one for each command line argument
83  //      ex: "nachos -d +" -> argv = {"nachos", "-d", "+"}
84  //----------------------------------------------------------------------
85
86  int
87  main(int argc, char **argv)
88  {
89      int argCount;           // the number of arguments
90                      // for a particular command
91
92      DEBUG('t', "Entering main");
93      (void) Initialize(argc, argv);
94
95      DLList dllist;
96      void *item;
97      int key = -1;
98
99      dllist.show();
100     // 当链表为空时，调用 Prepend
101     dllist.Prepend(new int[1]);
102     dllist.show();
103     // 当链表非空时，调用 Prepend
104     dllist.Prepend(new int[1]);
105     dllist.show();
106     // 当链表非空时，调用 Remove
107     item = dllist.Remove(&key);
108     printf("List remove key: %d, item: %p.\n", key, item);
109     dllist.show();
110     item = dllist.Remove(&key);
111     printf("List remove key: %d, item: %p.\n", key, item);
112     dllist.show();
113     // 当链表为空时，调用 Remove
```

```
    item = dllist.Remove(&key);
    printf("List remove key: %d, item: %p.\n", key, item);
    dllist.show();
    // 当链表为空时，调用 Append
    dllist.Append(new int[1]);
    dllist.show();
    // 当链表非空时，调用 Append
    dllist.Append(new int[1]);
    dllist.show();
    // 清空链表
    while(!dllist.IsEmpty()) dllist.Remove(&key);
    dllist.show();
    // 当链表为空时，调用 SortedInsert 插入
    dllist.SortedInsert(new int[1], 50);
    dllist.show();
    // 调用 SortedInsert，头插入
    dllist.SortedInsert(new int[1], 0);
    dllist.show();
    // 调用 SortedInsert，尾插入
    dllist.SortedInsert(new int[1], 100);
    dllist.show();
    // 调用 SortedInsert，中间插入
    dllist.SortedInsert(new int[1], 50);
    dllist.show();
    // 调用 SortedRemove，删除不存在的 key
    item = dllist.SortedRemove(key = 20);
    printf("List remove key: %d, item: %p.\n", key, item);
    dllist.show();
    // 调用 SortedRemove，中间删除
    item = dllist.SortedRemove(key = 50);
    printf("List remove key: %d, item: %p.\n", key, item);
    dllist.show();
    // 调用 SortedRemove，头删除
    item = dllist.SortedRemove(key = 0);
    printf("List remove key: %d, item: %p.\n", key, item);
    dllist.show();
    // 调用 SortedRemove，尾删除
    item = dllist.SortedRemove(key = 100);
    printf("List remove key: %d, item: %p.\n", key, item);
    dllist.show();
    // 清空链表
    while(!dllist.IsEmpty()) dllist.Remove(&key);
```

```
156     dllist.show();
157     // 调用 insert
158     insert(5, dllist);
159     dllist.show();
160     // 调用 remove
161     remove(5, dllist);
162     dllist.show();
163 #ifdef THREADS
164     // for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount) {
165     //   argCount = 1;
166     //   switch (argv[0][1]) {
167     //   case 'q':
168     //     testnum = atoi(argv[1]);
169     //     argCount++;
170     //     break;
171     //   default:
172     //     testnum = 1;
173     //     break;
174     //   }
175     // }
176 
177     // ThreadTest();
178 #endif
179 
180     for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount) {
181     argCount = 1;
182         if (!strcmp(*argv, "-z"))               // print copyright
183             printf (copyright);
184 #ifdef USER_PROGRAM
185         if (!strcmp(*argv, "-x")) {         // run a user program
186         ASSERT(argc > 1);
187             StartProcess(*(argv + 1));
188             argCount = 2;
189         } else if (!strcmp(*argv, "-c")) {      // test the console
190         if (argc == 1)
191             ConsoleTest(NULL, NULL);
192         else {
193         ASSERT(argc > 2);
194             ConsoleTest(*(argv + 1), *(argv + 2));
195             argCount = 3;
196         }
197         interrupt->Halt();      // once we start the console, then
```

```
                    // Nachos will loop forever waiting
                    // for console input
    }
#endif // USER_PROGRAM
#ifdef FILESYS
    if (!strcmp(*argv, "-cp")) {        // copy from UNIX to Nachos
        ASSERT(argc > 2);
        Copy(*(argv + 1), *(argv + 2));
        argCount = 3;
    } else if (!strcmp(*argv, "-p")) {  // print a Nachos file
        ASSERT(argc > 1);
        Print(*(argv + 1));
        argCount = 2;
    } else if (!strcmp(*argv, "-r")) {  // remove Nachos file
        ASSERT(argc > 1);
        fileSystem->Remove(*(argv + 1));
        argCount = 2;
    } else if (!strcmp(*argv, "-l")) {  // list Nachos directory
            fileSystem->List();
    } else if (!strcmp(*argv, "-D")) {  // print entire filesystem
            fileSystem->Print();
    } else if (!strcmp(*argv, "-t")) {  // performance test
            PerformanceTest();
    }
#endif // FILESYS
#ifdef NETWORK
        if (!strcmp(*argv, "-o")) {
        ASSERT(argc > 1);
            Delay(2);               // delay for 2 seconds
                        // to give the user time to
                        // start up another nachos
            MailTest(atoi(*(argv + 1)));
            argCount = 2;
        }
#endif // NETWORK
    }

    currentThread->Finish();    // NOTE: if the procedure "main"
                // returns, then the program "nachos"
                // will exit (as any other normal program
                // would).  But there may be other
                // threads on the ready list.  We switch
```

```
                // to those threads by saying that the
                // "main" thread is finished, preventing
                // it from returning.
    return(0);          // Not reached...
}
```

附件 5 yield.cc

```
#include "thread.h"

static int PLACE = -1;

extern Thread *currentThread;
extern void DEBUG(char flag, char *format, ...);

void YieldInit(int place)
{
    PLACE = place;
}

void YIELD(int place)
{
    if(place == PLACE || PLACE == 0)
    {
        DEBUG('e', "%s yield in %d.\n", currentThread->getName(), place);
        currentThread->Yield();
    }
}
```

附件 6 threadtest.cc

```
// threadtest.cc
//  Simple test case for the threads assignment.
//
//  Create two threads, and have them context switch
//  back and forth between themselves by calling Thread::Yield,
//  to illustratethe inner workings of the thread system.
//
// Copyright (c) 1992-1993 The Regents of the University of California.
// All rights reserved.  See copyright.h for copyright notice and
limitation
// of liability and disclaimer of warranty provisions.

#include <stdlib.h>
#include <time.h>
#include "copyright.h"
#include "system.h"
```

```
#include "dllist.h"
//----------------------------------------------------------------------
// SimpleThread
//
//  "which" is simply a number identifying the thread, for debugging
//  purposes.
//----------------------------------------------------------------------

extern void insert(int n, DLList &dllist);
extern void remove(int n, DLList &dllist);
extern void YIELD(int place);

static int N = 4, T = 2;
static DLList dllist;

void ThreadInit(int t, int n)
{
    T = t, N = n;
}

void SimpleThread(int which)
{
    DEBUG('e', "%s is running.\n", currentThread->getName());
    YIELD(60);
    insert(N, dllist);
    YIELD(61);
    remove(N, dllist);
    YIELD(62);
    DEBUG('e', "%s is finish.\n", currentThread->getName());
    dllist.show();
}

//----------------------------------------------------------------------
// ThreadTest
//  Invoke a test routine.
//----------------------------------------------------------------------

void ThreadTest()
{
    srand(time(0));
    Thread *threads[T];
    for (int i = 0; i < T; ++ i)
```

```
    {
        char* name = new char[20];
        name[0] = '\0';
        strcat(name, "thread ");
        sprintf(name + 7, "%d", i);
        threads[i] = new Thread(name);
        threads[i]->Fork(SimpleThread, i);
    }
}
```

附件 7 dllist.cc

```
#include <iostream>
#include "dllist.h"
#include "thread.h"
#define KEY 50

extern Thread *currentThread;
extern void YIELD(int place);
extern void DEBUG(char flag, char *format, ...);

DLLElement::DLLElement(void * itemPtr, int sortKey)
{
    YIELD(1);
    next = NULL;
    YIELD(2);
    prev = NULL;
    YIELD(3);
    key = sortKey;
    YIELD(4);
    item = itemPtr;
    YIELD(5);
}

DLList::DLList()
{
    first = NULL;
    last = NULL;
}

DLList::~DLList()
{
    int key;
    while(!IsEmpty())
    {
```

```
34          Remove(&key);
35      }
36  }
37
38  void DLList::Prepend(void *item)
39  {
40      if(IsEmpty())
41      {
42          DLLElement *elm = new DLLElement(item, KEY);
43          first = elm;
44          last = elm;
45      }
46      else
47      {
48          DLLElement *elm = new DLLElement(item, first->key - 1);
49          elm->next = first;
50          first->prev = elm;
51          first = elm;
52      }
53  }
54
55  void DLList::Append(void *item)
56  {
57
58      if(IsEmpty())
59      {
60          DLLElement *elm = new DLLElement(item, KEY);
61          first = elm;
62          last = elm;
63      }
64      else
65      {
66          DLLElement *elm = new DLLElement(item, last->key + 1);
67          elm->prev = last;
68          last->next = elm;
69          last = elm;
70      }
71  }
72
73  void *DLList::Remove(int *keyPtr)
74  {
75      YIELD(6);
```

```
76      if(IsEmpty())
77      {
78          YIELD(7);
79          *keyPtr = -1;
80          YIELD(8);
81          DEBUG('e', "[ERROR]%s : list is empty!\n",
82  currentThread->getName());
83          return NULL;
84      }
85      else
86      {
87          YIELD(9);
88          void *item = first->item;
89          YIELD(10);
90          *keyPtr = first->key;
91          YIELD(11);
92          DLLElement *temp = first->next;
93          YIELD(12);
94          delete first;
95          YIELD(13);
96          if(temp == NULL)
97          {
98              YIELD(14);
99              last = NULL;
100         }
101         else
102         {
103             YIELD(15);
104             temp->prev = NULL;
105         }
106         YIELD(16);
107         first = temp;
108         YIELD(17);
109         return item;
110     }
111 }
112
113 bool DLList::IsEmpty()
114 {
115     YIELD(18);
116     if(first == NULL)
117     {
```

```
        YIELD(19);
        if(last == NULL)
        {
            YIELD(20);
            return true;
        }
        DEBUG('e', "[ERROR]first is NULL, last is not NULL!\n");
        YIELD(21);
    }
    else if(last == NULL)
    {
        DEBUG('e', "[ERROR]first is not NULL, last is NULL!\n");
    }
    YIELD(22);
    return false;
}

void DLList::SortedInsert(void *item, int sortKey)
{
    YIELD(23);
    DLLElement *elm = new DLLElement(item, sortKey);
    YIELD(24);
    if(IsEmpty())
    {
        YIELD(25);
        first = elm;
        YIELD(26);
        last = elm;
        YIELD(27);
    }
    else
    {
        YIELD(28);
        DLLElement *ptr = first;
        YIELD(29);
        while(ptr != NULL && ptr->key < sortKey)
        {
            YIELD(30);
            ptr = ptr->next;
            YIELD(31);
        }
        YIELD(32);
```

```
160         if(ptr == first) // insert in the head
161         {
162             YIELD(33);
163             elm->next = first;
164             YIELD(34);
165             first->prev = elm;
166             YIELD(35);
167             first = elm;
168             YIELD(36);
169         }
170         else if(ptr == NULL) // insert in the tail
171         {
172             YIELD(37);
173             elm->prev = last;
174             YIELD(38);
175             last->next = elm;
176             YIELD(39);
177             last = elm;
178             YIELD(40);
179         }
180         else
181         {
182             YIELD(41);
183             elm->next = ptr;
184             YIELD(42);
185             ptr->prev->next = elm;
186             YIELD(43);
187             elm->prev = ptr->prev;
188             YIELD(44);
189             ptr->prev = elm;
190             YIELD(45);
191         }
192     }
193 }
194 
195 void *DLList::SortedRemove(int sortKey)
196 {
197     DLLElement *ptr = first;
198     while(ptr != NULL && ptr->key != sortKey)
199     {
200         ptr = ptr->next;
201     }
```

```
202     if(ptr == NULL)
203     {
204         return NULL;
205     }
206     void *item = ptr->item;
207     if(ptr == first)
208     {
209         first = first->next;
210         if(first == NULL) // list is empty
211         {
212             last = NULL;
213         }
214         else
215         {
216             first->prev = NULL;
217         }
218     }
219     else if(ptr == last)
220     {
221         last = last->prev;
222         if(last == NULL) // list is empty
223         {
224             first = NULL;
225         }
226         else
227         {
228             last->next = NULL;
229         }
230     }
231     else
232     {
233         ptr->next->prev = ptr->prev;
234         ptr->prev->next = ptr->next;
235     }
236     delete ptr;
237     return item;
238 }
239
240 void DLList::show()
241 {
242     if(IsEmpty())
243     {
```

```
        printf("List is empty!\n");
    }
    else
    {
        DLLElement *tmp = first;
        printf("List from head to tail:\n");
        while(tmp != NULL)
        {
            printf("key: %d, item: %p\n", tmp->key, tmp->item);
            tmp = tmp->next;
        }
        tmp = last;
        printf("List from tail to head:\n");
        while(tmp != NULL)
        {
            printf("key: %d, item: %p\n", tmp->key, tmp->item);
            tmp = tmp->prev;
        }
    }
}
```

附件 8 dllist-driver.cc

```
#include <stdlib.h>
#include <time.h>
#include "dllist.h"
#include "thread.h"

#define MAX_KEY 100

extern void YIELD(int place);
extern Thread *currentThread;
extern void DEBUG(char flag, char *format, ...);

void insert(int n, DLList &dllist)
{
    YIELD(50);
    for(int i = 0; i < n; ++ i)
    {
        void *item = new int[1];
        int key = rand() % MAX_KEY;
        YIELD(51);
        dllist.SortedInsert(item, key);
        DEBUG('e', "%s: insert key:%2d, item:%p.\n",
currentThread->getName(), key, item);
```

```
        YIELD(52);
    }
    YIELD(53);
}

void remove(int n, DLList &dllist)
{
    YIELD(54);
    for(int i = 0; i < n; ++ i)
    {
        int key = -1;
        YIELD(55);
        void *item = dllist.Remove(&key);
        DEBUG('e', "%s: remove key:%2d, item:%p.\n",
currentThread->getName(), key, item);
        YIELD(56);
    }
    YIELD(57);
}
```

附件 9 main.cc

```
// main.cc
//  Bootstrap code to initialize the operating system kernel.
//
//  Allows direct calls into internal operating system functions,
//  to simplify debugging and testing.  In practice, the
//  bootstrap code would just initialize data structures,
//  and start a user program to print the login prompt.
//
//  Most of this file is not needed until later assignments.
//
// Usage: nachos -d <debugflags> -rs <random seed #>
//      -s -x <nachos file> -c <consoleIn> <consoleOut>
//      -f -cp <unix file> <nachos file>
//      -p <nachos file> -r <nachos file> -l -D -t
//              -n <network reliability> -m <machine id>
//              -o <other machine id>
//              -z
//
//    -d causes certain debugging messages to be printed (cf. utility.h)
//    -rs causes Yield to occur at random (but repeatable) spots
//    -z prints the copyright message
//
//  USER_PROGRAM
```

```
//    -s causes user programs to be executed in single-step mode
//    -x runs a user program
//    -c tests the console
//
//  FILESYS
//    -f causes the physical disk to be formatted
//    -cp copies a file from UNIX to Nachos
//    -p prints a Nachos file to stdout
//    -r removes a Nachos file from the file system
//    -l lists the contents of the Nachos directory
//    -D prints the contents of the entire file system
//    -t tests the performance of the Nachos file system
//
//  NETWORK
//    -n sets the network reliability
//    -m sets this machine's host id (needed for the network)
//    -o runs a simple test of the Nachos network software
//
//  NOTE -- flags are ignored until the relevant assignment.
//  Some of the flags are interpreted here; some in system.cc.
//
// Copyright (c) 1992-1993 The Regents of the University of California.
// All rights reserved.  See copyright.h for copyright notice and
limitation
// of liability and disclaimer of warranty provisions.

#define MAIN
#include "copyright.h"
#undef MAIN

#include "utility.h"
#include "system.h"
#include "dllist.h"

#ifdef THREADS
extern int testnum;
#endif

// External functions used by this file

extern void ThreadTest(void), Copy(char *unixFile, char *nachosFile);
extern void Print(char *file), PerformanceTest(void);
```

```
66  extern void StartProcess(char *file), ConsoleTest(char *in, char *out);
67  extern void MailTest(int networkID);
68
69  extern void insert(int n, DLList &dllist);
70  extern void remove(int n, DLList &dllist);
71
72  extern void YieldInit(int place);
73  extern void YIELD(int place);
74  extern void ThreadInit(int t, int n);
75
76  //----------------------------------------------------------------------
77  // main
78  //  Bootstrap the operating system kernel.
79  //
80  //  Check command line arguments
81  //  Initialize data structures
82  //  (optionally) Call test procedure
83  //
84  //  "argc" is the number of command line arguments (including the name
85  //      of the command) -- ex: "nachos -d +" -> argc = 3
86  //  "argv" is an array of strings, one for each command line argument
87  //      ex: "nachos -d +" -> argv = {"nachos", "-d", "+"}
88  //----------------------------------------------------------------------
89
90  int main(int argc, char **argv)
91  {
92      int argCount; // the number of arguments
93                    // for a particular command
94
95      DEBUG('t', "Entering main");
96      (void)Initialize(argc, argv);
97  #ifdef THREADS
98      for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount)
99      {
100         argCount = 1;
101         if (!strcmp(*argv, "-q"))
102         {
103             if(argc >= 3 && argv[1][0] != '-')
104             ThreadInit(atoi(argv[1]), atoi(argv[2]));
105             argCount += 2;
106         }
107         else if(!strcmp(*argv, "-y"))
```

```
        {
            YieldInit(atoi(argv[1]));
            ++ argCount;
        }
    }
    ThreadTest();
#endif

    for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount)
    {
        argCount = 1;
        if (!strcmp(*argv, "-z")) // print copyright
            printf(copyright);
#ifdef USER_PROGRAM
        if (!strcmp(*argv, "-x"))
        { // run a user program
            ASSERT(argc > 1);
            StartProcess(*(argv + 1));
            argCount = 2;
        }
        else if (!strcmp(*argv, "-c"))
        { // test the console
            if (argc == 1)
                ConsoleTest(NULL, NULL);
            else
            {
                ASSERT(argc > 2);
                ConsoleTest(*(argv + 1), *(argv + 2));
                argCount = 3;
            }
            interrupt->Halt(); // once we start the console, then
                               // Nachos will loop forever waiting
                               // for console input
        }
#endif // USER_PROGRAM
#ifdef FILESYS
        if (!strcmp(*argv, "-cp"))
        { // copy from UNIX to Nachos
            ASSERT(argc > 2);
            Copy(*(argv + 1), *(argv + 2));
            argCount = 3;
        }
```

```
150         else if (!strcmp(*argv, "-p"))
151         { // print a Nachos file
152             ASSERT(argc > 1);
153             Print(*(argv + 1));
154             argCount = 2;
155         }
156         else if (!strcmp(*argv, "-r"))
157         { // remove Nachos file
158             ASSERT(argc > 1);
159             fileSystem->Remove(*(argv + 1));
160             argCount = 2;
161         }
162         else if (!strcmp(*argv, "-l"))
163         { // list Nachos directory
164             fileSystem->List();
165         }
166         else if (!strcmp(*argv, "-D"))
167         { // print entire filesystem
168             fileSystem->Print();
169         }
170         else if (!strcmp(*argv, "-t"))
171         { // performance test
172             PerformanceTest();
173         }
174 #endif // FILESYS
175 #ifdef NETWORK
176         if (!strcmp(*argv, "-o"))
177         {
178             ASSERT(argc > 1);
179             Delay(2); // delay for 2 seconds
180                       // to give the user time to
181                       // start up another nachos
182             MailTest(atoi(*(argv + 1)));
183             argCount = 2;
184         }
185 #endif // NETWORK
186     }
187 
188     currentThread->Finish(); // NOTE: if the procedure "main"
189                              // returns, then the program "nachos"
190                              // will exit (as any other normal program
191                              // would).  But there may be other
```

```
192                             // threads on the ready list.  We switch
193                             // to those threads by saying that the
194                             // "main" thread is finished, preventing
195                             // it from returning.
196     return (0); // Not reached...
197 }
```